Real-Time Dashboard with Kafka, Spark, MongoDB and Streamlit

Khawla BOUGHATTAS
3 IDSD - ID

1. Introduction

In recent years, the volume and velocity of data generated by applications, users, and connected systems have increased significantly. Traditional batch processing systems are no longer sufficient when organizations need instant insights and real-time monitoring. This project focuses on designing and understanding a real-time data processing pipeline that enables continuous data ingestion, processing, storage, and visualization.

The project is based on an existing GitHub repository titled “Building a Real-time Dashboard with Kafka, Spark, MongoDB and Streamlit”. The repository was used as a starting point to study, execute, and understand a complete end-to-end real-time architecture. The main objective is not only to run the project, but also to deeply analyze its architecture, components, workflow, challenges, and learning outcomes. This project was implemented using Docker to ensure reproducibility, portability, and ease of deployment.

2. Motivation and Problem Statement

2.1 Motivation

Many modern systems (e-commerce platforms, IoT applications, financial systems, social media platforms) generate data continuously.

Decision-makers often require up-to-date dashboards to:

- Monitor system activity
- Track user behavior
- Detect anomalies or abnormal patterns
- Take immediate actions

Batch processing introduces latency, meaning that insights are always delayed. This motivates the need for real-time data streaming architectures.

2.2 Problem Statement

The main problem addressed in this project is:

> How can we design and implement a scalable real-time data pipeline that collects streaming data, processes it in real time, stores the results efficiently, and visualizes them through an interactive dashboard?

This problem involves several technical challenges:

- Managing high-frequency streaming data
- Processing data continuously with low latency

- Ensuring fault tolerance and scalability

- Providing a clear and interactive visualization layer

3. Why Using a GitHub Project as a Base

Instead of building the entire system from scratch, this project starts from an existing GitHub repository. This decision was intentional and justified for several reasons:

1. Educational Value: The repository provides a complete and realistic real-time architecture that is commonly used in industry.
2. Time Efficiency: Starting from a functional base allows focusing on understanding concepts rather than setup difficulties.
3. Best Practices: The project demonstrates good practices in streaming, data processing, and visualization.
4. Reproducibility: The solution is already tested and can be reproduced easily.
5. Customization Potential: The project can be modified and extended to explore new ideas and improvements.

The approach followed was: - Clone the repository - Analyze each component separately - Understand data flow and interactions - Execute the project step by step - Identify challenges and limitations

4. Technologies Overview and Definitions

4.1 Apache Kafka

Apache Kafka is a distributed event streaming platform used to build real-time data pipelines. It is designed to handle high-throughput, fault-tolerant, and scalable data streams.

Key Concepts:

- Producer: Sends messages to Kafka topics

- Consumer: Reads messages from Kafka topics

- Topic: A logical channel where messages are stored

- Broker: A Kafka server

In this project, Kafka acts as a buffer and transport layer between data generation and processing.

4.2 Apache Spark Structured Streaming

Apache Spark Structured Streaming is a real-time data processing engine that allows processing streaming data using the same APIs as batch processing.

Main Characteristics:

- Micro-batch processing
- Fault tolerance using checkpointing
- Scalability across clusters

In this project, Spark reads data from Kafka, applies transformations and aggregations, and writes the processed results to MongoDB.

4.3 MongoDB

MongoDB is a NoSQL document-oriented database. It stores data in flexible JSON-like documents, making it suitable for dynamic and semi-structured data.

Why MongoDB?

- High write performance
- Flexible schema
- Easy integration with Python and Spark

MongoDB is used to store the processed streaming data so it can be easily queried by the dashboard.

4.4 Streamlit

Streamlit is a Python-based framework for building interactive data dashboards quickly.

Advantages:

- Simple and fast development
- Native integration with Python
- Supports real-time visualization

In this project, Streamlit is used to display charts, tables, and metrics based on data retrieved from MongoDB.

4.5 Docker & Docker Compose

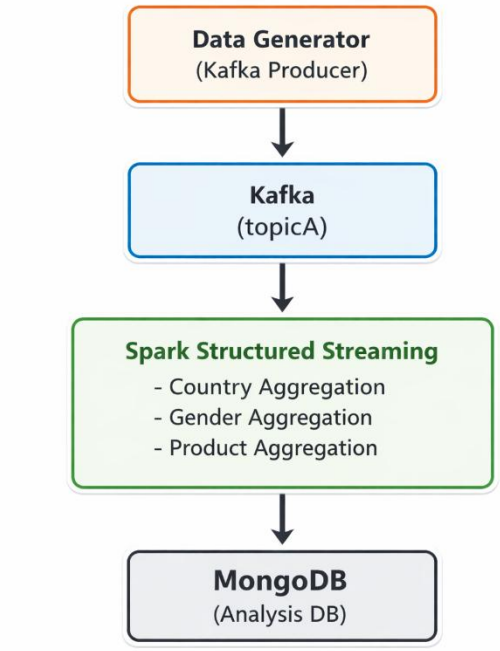
Docker is used to:

- Containerize all services
- Avoid environment conflicts
- Simplify deployment

5. System Architecture

5.1 Global Architecture

The architecture follows a layered streaming pipeline:



5.2 Component Roles

Data Generator

- Implemented in Python
- Generates fake data using the Faker library
- Sends JSON messages continuously to Kafka

Kafka

- Receives streaming data
- Temporarily stores messages in topics

- Ensures reliable delivery to Spark

Spark Structured Streaming

- Reads messages from Kafka topics
- Parses and transforms JSON data
- Performs aggregations (counts, sums, statistics)
- Writes results to MongoDB

MongoDB

- Stores processed data in collections
- Acts as a persistence layer

Streamlit Dashboard

- Queries MongoDB collections
- Displays real-time charts and tables
- Updates data dynamically

6. Data Flow and Workflow

1. The data generator produces events continuously
2. Kafka receives and buffers these events
3. Spark reads events in micro-batches
4. Data is transformed and aggregated
5. Processed data is stored in MongoDB
6. Streamlit retrieves data and visualizes it

This workflow ensures a smooth and continuous flow of real-time data from source to visualization.

7. Implementation Details

7.1 Data Generation

Fake user or event data is generated using Python and sent to Kafka using a Kafka producer. Each message is formatted as JSON.

7.2 Spark Processing

Spark Structured Streaming:

- Reads Kafka messages

- Converts binary values to JSON

- Applies transformations and aggregations

- Uses checkpointing for fault tolerance

7.3 Data Storage

MongoDB stores the aggregated data in collections, allowing fast querying and flexible schema management.

7.4 Dashboard Visualization

Streamlit:

- Connects to MongoDB
- Loads data into Pandas DataFrames
- Displays charts and tables
- Refreshes data periodically

8. Challenges and Difficulties

8.1 Kafka Integration with Spark

- Kafka configuration and topic management
- Missing Kafka connector caused errors like: Failed to find data source: kafka
- Managing multiple services simultaneously
- Dependency Conflicts :
    - Python 3.12 compatibility issues with kafka-python
    - Missing libraries such as six

8.2 MongoDB Authentication

- Incorrect URI formats caused permission errors
- Streamlit dubagging : Hard to debug infinite streams & Required careful log analysis

9. Solutions and Improvements

- Manually added Kafka Spark connector JAR
- Used correct versions compatible with Spark 3.5.7
- Switched to stable Kafka client versions
- Verified Docker network communication
- Used checkpointing for stream reliability

10. Results and Outcomes

- Successful execution of a real-time data pipeline
- Functional interactive dashboard
- Clear understanding of streaming architectures
- Hands-on experience with distributed systems

11. Learning Outcomes

Through this project, the following skills were developed:

- Understanding real-time data pipelines
- Working with Kafka and Spark
- Integrating NoSQL databases
- Building real-time dashboards
- Debugging distributed systems

12. Limitations and Future Work

Limitations

- Dashboard scalability is limited for very large datasets
- Single-node execution environment

Future Enhancements

- Implement windowed aggregations (time-based)
- Add schema validation
- Improve fault tolerance and monitoring
- Deploy on cloud platforms (AWS, GCP)

13. Conclusion

This project demonstrates a complete real-time data processing pipeline using Kafka, Spark, MongoDB, and Streamlit. By starting from an existing GitHub repository and deeply analyzing its architecture and implementation, valuable insights were gained into modern streaming systems. The project provides a strong foundation for building scalable real-time analytics platforms and can be extended for real-world applications.